

Experiment I

LOGIN FORM

1. Start the MainActivity.
2. Set the application's user interface using the activity_main.xml layout.
3. Find and Store references to the UI components:
 - username (EditText with ID R.id.uname)
 - password (EditText with ID R.id.pwd)
 - login (Button with ID R.id.lgn)
4. Define the VALID_USERNAME as "user" and VALID_PASSWORD as "pass".
5. Attach an OnClickListener to the login button.
6. Wait for user interaction.
7. When the login button is clicked:
8. Read the text from the username field, convert it to a String, and remove whitespace, storing it as enteredUname.
9. Read the text from the password field, convert it to a String, and remove whitespace, storing it as enteredPass.
10. Check if enteredUname is empty OR enteredPass is empty:
 - If True:
 - Display a Toast message: "Please enter both details..".
 - Goto Step 7 (Wait for next click).
11. Else If (If both fields are not empty), CALL the isValid(enteredUname, enteredPass) function.
 - The isValid function returns TRUE if enteredUname equals "user" AND enteredPass equals "pass".
 - If isValid returns TRUE:
 - Display a Toast message: "Login Success...".
 - Goto Step 7 (Wait for next click).
12. Else (If fields are not empty but credentials are incorrect):
 - Display a Toast message: "Invalid Credentials...".
 - Goto Step 7 (Wait for next click).

Auxiliary Function: isValid(euname, epass)

- a. Input: String euname, String epass.
- b. Return (euname equals "user") AND (epass equals "pass").

Auxiliary Function: showToast(msg)

- a. Input: String msg.
- b. Display a long-duration Toast message containing the text in msg.
- c. Return.

Experiment II

SIMPLE CALCULATOR

1. Start the application and load the UI layout (activity_main.xml).
2. Find and Store a reference to the display area (textView1).
3. Initialize the state variables:
 - currentInput (String) = "" (Empty string)
 - operand1 (Double) = 0
 - operator (String) = "" (Empty string)
4. The application then Waits for a button click event.
5. Digit Button Click (onDigitClick): Triggered when any number (0-9) or the decimal point button is pressed.
 - Input: The text of the clicked button (e.g., "7", "5", ".").
 - Append the button's text to the currentInput string.
 - Update Display: Call updateDisplay() to show the new currentInput in textView1.
6. Operator Button Click (onOperatorClick): Triggered when '+', '-', '*', or '/' is pressed.
 - Check if currentInput is NOT empty.
 - If Not Empty:
 - a. Convert the currentInput string to a Double and Store it in operand1.
 - b. Store the clicked button's text (the operator) in the operator variable.
 - c. Reset currentInput to "" (Empty string), preparing for the entry of the second operand.
7. Equals Button Click (onEqualsClick): Triggered when '=' is pressed.
 - Check if currentInput is NOT empty (meaning the second operand has been entered).
 - If Not Empty:
 - a. Convert the currentInput string to a Double and STORE it as operand2.
 - b. Call the performOperation(operand1, operand2, operator) function to calculate the result.
 - c. Convert the calculated result (Double) back into a String.
 - d. Store this String result in currentInput.
 - Update Display: Call updateDisplay() to show the final result in textView1.

8. Clear Button Click (onClearClick): Triggered when 'C' is pressed.

- Reset all state variables:
 - `currentInput = ""`
 - `operand1 = 0`
 - `operator = ""`
- Update Display: Call `updateDisplay()` (which will effectively show an empty string or "0" depending on default display behavior).

Auxiliary Function: `performOperation(op1, op2, op)`

Input: Double `op1` (First Operand), Double `op2` (Second Operand), String `op` (Operator).

Switch based on the value of `op`:

- Case "+": RETURN `op1+op2`.
- Case "-": RETURN `op1-op2`.
- Case "*": RETURN `op1*op2`.
- Case "/":
 - If `op2 = 0`: RETURN `op1/op2`.
 - ELSE: RETURN `Double.NaN` (Not a Number, handling division by zero).
- Default: RETURN 0.

Experiment III

ACTIVITY LIFECYCLE

1. Start the MainActivity.
2. Call the system method → onCreate(Bundle savedInstanceState):
 - a. Load the UI layout (activity_main.xml).
 - b. Show Toast: "Activity Created".
 - c. Find and Store references to the six lifecycle buttons.
 - d. Attach an OnClickListener to each of the six buttons.
3. Call the system method → onStart():
 - a. Show Toast: "Activity Started".
4. Call the system method → onResume():
 - a. Show Toast: "Activity Resumed".
5. Wait for user interaction or system events (e.g., pressing the home button, receiving a phone call, rotating the device).
6. WHEN a lifecycle button is clicked:
 - a. Retrieve the designated lifecycle stage from the button's click handler.
 - b. Show Toast: Display the corresponding message.

Auxiliary Function: showToast(msg)

- a. Input: String msg.
- b. Display a short-duration Toast message containing the text in msg.
- c. Return.

Experiment IV

VALIDATIONS ON UI CONTROLS

1. Start the MainActivity.
2. Load the user interface defined in activity_main.xml.
3. Find and Store references for all six Button elements (ConstraintLayout, LinearLayout, GridLayout, RelativeLayout, FrameLayout, and TableLayout).
4. Create a new anonymous View.OnClickListener object named buttonClickListener. This object contains the logic for what happens when any of the attached buttons are clicked.
5. Assign the same buttonClickListener to the setOnClickListener method for all six buttons.
6. Wait for a user button press.
7. Button Click Event (onClick method within buttonClickListener)
8. When any of the six layout buttons is clicked:
 - a. Identify the clicked view (view) as a Button.
 - b. Retrieve the text content from the clicked Button, storing it as the layoutName String.
 - c. Call the private function displayToken(layoutName).

Auxiliary Function: displayToken(layoutName)

- a. Input: String layoutName.
- b. Construct the Toast message string: "Token from " + layoutName.
- c. Display a short-duration Toast message containing the constructed string.
- d. Return to the waiting state.

Experiment V

FACEBOOK PAGE

1. Start the MainActivity.
2. Load the user interface defined in activity_main.xml.
3. Find and Store references for the three interactive ImageView components:
 - likeImageView (R.id.likeView)
 - commentImageView (R.id.cmmntView)
 - shareImageView (R.id.shareView)
4. Attach a new OnClickListener to the likeImageView.
5. This listener is configured to execute → showToast("You Clicked Like Button") when the likeImageView is pressed.
6. Attach a new OnClickListener to the commentImageView.
7. This listener is configured to execute → showToast("You Clicked Comment Button") when the commentImageView is pressed.
8. Attach a new OnClickListener to the shareImageView.
9. This listener is configured to execute → showToast("You Clicked Share Button") when the shareImageView is pressed.
10. Wait for user interaction.
11. When the user taps one of the interactive image buttons:
12. The OnClickListener attached to that specific button is triggered.
13. Execute the showToast function with the pre-defined message corresponding to the clicked button.
14. The application returns to the waiting state.

Auxiliary Function: showToast(message)

- a. Input: String message.
- b. Display a short-duration Toast message containing the text in message.
- c. Return.

Experiment VI

IMAGE TOGGLE

1. Start the MainActivity.
2. Load the user interface defined in activity_main.xml.
3. Find and Store references for the two ImageView components:
 - i1 (R.id.img1)
 - i2 (R.id.img2)
4. Set the initial state (based on activity_main.xml):
 - i1 is VISIBLE.
 - i2 is GONE (hidden).
5. Assign the Activity itself as the OnClickListener for both i1 and i2.
6. Wait for a user click event.
7. When the user taps either i1 or i2:
8. Identify which image was clicked by checking the view's ID.
9. If the clicked view's ID Equals R.id.img1 (Image 1 was clicked):
 - a. Set i1's visibility to GONE (hide Image 1).
 - b. Set i2's visibility to VISIBLE (show Image 2).
10. Else:
 - a. Set i1's visibility to VISIBLE (show Image 1).
 - b. Set i2's visibility to GONE (hide Image 2).
11. Return to the waiting state.

Experiment VII

INTENTS AND SHAREDREFERENCES

1. Start the MainActivity.
2. Load the UI layout (activity_main.xml).
3. Find and Store references for the three EditText fields and the Button:
 - username (R.id.usernameText)
 - email (R.id.emailText)
 - passwd (R.id.passwordText)
 - regbtn (R.id.registerButton)
4. Attach an OnClickListener to the regbtn (Register Button).
5. Wait for the user to enter data and click the Register button.
6. When the regbtn is clicked:
7. Read the content from each EditText field and store it as String variables:
 - uname = username.getText().toString()
 - em = email.getText().toString()
 - pwd = passwd.getText().toString()
8. Get a SharedPreferences object named "MyPref" in MODE_PRIVATE.
9. Get an Editor object to modify the preferences.
10. Save Data:
 - a. Put the value of uname into the key "username".
 - b. Put the value of em into the key "email".
 - c. Put the value of pwd into the key "password".
 - d. Apply the changes to save the data asynchronously.
11. Display a brief Toast message: "Registration Succesful".
12. Restart Activity:
 - Create a new Intent to launch the MainActivity (itself).
 - Start the new Activity.
13. Loop: Since the MainActivity restarts itself (Step 12), the algorithm JUMPS BACK to Step 1 (Initialization) for the new instance of the Activity.

Experiment VIII

ARRAYADAPTER WITH LISTVIEW

1. Start the MainActivity.
2. Load the application's user interface defined in activity_main.xml.
3. Find and Store a reference to the ListView element using its ID (R.id.listView).
4. Create a String array named fruits containing the following list:
 - "Apple", "Banana", "Cherry", "Dates", "Grape", "Kiwi", "Lemon", "Mango", "Orange", "Peach"
5. Create an ArrayAdapter (adapter) using the following parameters:
 - a. Context: The current Activity (this).
 - b. Layout Resource: A standard Android layout for a single item (android.R.layout.simple_list_item_1).
 - c. Data Source: The fruits array.
6. The ArrayAdapter's purpose is to act as a bridge, taking the data (fruits) and formatting it according to the specified layout.
7. Set the created adapter as the data source for the listView using listView.setAdapter(adapter).
8. The ListView now displays the list of fruit names, with each item rendered using the simple list item layout.

Experiment IX

OPTIONS MENU

1. Start the MainActivity.
2. Load the base layout (activity_main.xml), which displays the "Main Activity" text.
3. Wait for the Android system to request the creation of the options menu.
4. When the system calls onCreateOptionsMenu:
5. Load the menu resource file (R.menu.menu_main) into the action bar.
6. The menu is populated with two items:
 - Item 1: "Go to Activity one" (ID: R.id.action_activity_one)
 - Item 2: "Go to Activity two" (ID: R.id.action_activity_two)
7. Display the menu items.
8. When the user selects one of the menu items:
9. Retrieve the ID of the selected MenuItem.
10. Check the item ID:
11. If ID equals R.id.action_activity_one:
 - a. Create an Intent targeting the activityOne.class.
 - b. Start the new activityOne.
 - c. Show Toast: "Navigating to Activity One".
 - d. Mark the event as handled (return true).
12. Else If ID equals R.id.action_activity_two:
 - a. Create an Intent targeting the activityTwo.class.
 - b. Start the new activityTwo.
 - c. Show Toast: "Navigating to Activity Two".
 - d. Mark the event as handled (return true).
13. Else (if any other item is clicked):
14. Delegate the handling to the parent class.
15. When activityOne is started: Load the activity_one.xml layout, which displays "This is Activity One".
16. When activityTwo is started: Load the activity_two.xml layout, which displays "This is Activity Two".

Experiment X

EXPLICIT INTENTS

1. Start the MainActivity.
2. Load the UI layout (activity_main.xml).
3. Find and Store a reference to the navigateButton (R.id.navigateButton).
4. Attach a new OnClickListener to the navigateButton.
5. Wait for the user to click the button.
6. When the navigateButton is clicked:
7. Create an Explicit Intent (explicitIntent).
8. The intent specifies the current context (MainActivity.this) and explicitly names the target component: SecondActivity.class.
9. Call startActivity(explicitIntent): This command tells the Android system to launch the SecondActivity.
10. The application flow immediately moves to the SecondActivity initialization.
11. Start the SecondActivity.
12. Load the UI layout (activity_second.xml).
13. Display the text "Welcome to second activity".
14. The SecondActivity is now running and visible to the user.

Experiment XI

SPINNER COMPONENT

1. Start the MainActivity.
2. Load the UI layout (activity_main.xml), which displays a TextView and the Spinner.
3. Find and Store a reference to the Spinner element (R.id.languageSpinner).
4. Create an ArrayAdapter (adapter) by loading the data from the string array resource: R.array.languages_array (which contains: Java, Python, C++, etc.).
5. The ArrayAdapter uses the standard Android layout (android.R.layout.simple_spinner_item) for how the selected item appears in the closed Spinner.
6. Set the dropdown view resource to android.R.layout.simple_spinner_dropdown_item for how the items look when the Spinner is expanded.
7. Assign the created adapter as the data source for the Spinner.
8. Attach an AdapterView.OnItemSelectedListener to the Spinner.
9. The application now WAITS for the user to interact with the Spinner.
10. When the user selects an item from the Spinner's dropdown list:
11. Retrieve the selected item's text using parent.getItemAtPosition(position).toString(), storing it as selectedItem.
12. Construct the Toast message string: "selected " + selectedItem.
13. Display a short-duration Toast message containing the constructed string (e.g., "selected Python").
14. Return to the waiting state.

Experiment XII

FRAGMENTS

1. Start the MainActivity.
2. Load the UI layout (activity_main.xml), which contains a container (FrameLayout) and two control buttons.
3. Find and Store references for the two Button elements:
 - button1 (R.id.button_fragment1)
 - button2 (R.id.button_fragment2)
4. Instantiate the two Fragment objects:
 - fragment1 = new FragmentOne()
 - fragment2 = new FragmentTwo()
5. Assign the Activity itself (this) as the OnClickListener for both button1 and button2.
6. Call loadFragment(fragment1) to display FragmentOne immediately upon startup.
7. Wait for a user click event.
8. When a Fragment is loaded, its lifecycle begins.
9. Call the onCreateView method for the specific Fragment.
10. Inflate the corresponding layout file:
 - FragmentOne loads fragment_one.xml (Red background, "This is Fragment One").
 - FragmentTwo loads fragment_two.xml (Blue background, "This is Fragment Two").
11. The Fragment's view is then RETURNED and placed inside the fragment_container.
12. Get the FragmentManager from the support library.
13. Begin a FragmentTransaction.
14. Replace the current Fragment inside the R.id.fragment_container with the new fragment passed as the argument.
15. Commit the transaction, executing the Fragment change.
16. When a user taps either button:
17. Identify the ID of the clicked view (viewId).
18. If viewId EQUALS R.id.button_fragment1:
 - CALL loadFragment(fragment1).

19. Else If viewId EQUALS R.id.button_fragment2:

- CALL loadFragment(fragment2).

20. Return to the waiting state.

Experiment XIII

SQLITE DATABASE

1. Start the MainActivity.
2. Load the UI layout (activity_main.xml).
3. Instantiate the DatabaseHelper object (myDb).
4. Find and Store references for the three EditText fields (editTextId, editTextName, editTextEmail).
5. Find and Store references for the four Button elements (btnAdd, btnView, btnUpdate, btnDelete).
6. Call all four data operation setup functions: addData(), viewAll(), updateData(), and deleteData().
7. Wait for a user button click.
8. When **btnAdd** is clicked:
 - a. Read Name and Email from their respective EditText fields.
 - b. Call myDb.insertData(Name, Email).
 - c. Check the boolean result (isInserted):
 - d. If TRUE: SHOW TOAST: "Data Inserted Successfully!".
 - e. If FALSE: SHOW TOAST: "Data Insertion Failed".
9. When **btnView** is clicked:
 - a. Call myDb.getAllData() to get a Cursor object (res).
 - b. Check if res.getCount() == 0:
 - c. If TRUE: CALL showMessage("Error", "Nothing found").
 - d. If FALSE:
 - Initialize an empty StringBuffer (buffer).
 - Iterate through the Cursor using res.moveToNext():
 - Append ID, Name, and Email data to the buffer string, separated by newlines.
 - e. Call showMessage("Data", buffer.toString()).
10. When **btnUpdate** is clicked:
 - a. Read ID, Name, and Email from their respective EditText fields.
 - b. Call myDb.updateData(ID, Name, Email).
 - c. Check the boolean result (isUpdated):

- d. If TRUE: SHOW TOAST: "Data Updated Successfully!".
- e. If FALSE: SHOW TOAST: "Data Update Failed".

11. When **btnDelete** is clicked:

- a. Read ID from its EditText field.
- b. Call myDb.deleteData(ID) to get the count of deleted rows (deletedRows).
- c. Check if deletedRows > 0:
- d. If TRUE: SHOW TOAST: "Data Deleted Successfully!".
- e. If FALSE: SHOW TOAST: "Data Deletion Failed".

Auxiliary Function: showMessage

- a. Input: String title, String message.
- b. Create a new AlertDialog.Builder.
- c. Set cancelable to true.
- d. Set the dialog's Title and Message.
- e. Show the AlertDialog to the user.